RLA PROJECT #2

by Kevin Otah Ogbusuo, Mishvaba Hitensinh Vaghela,
Pritheesh Selvaraja Kumar
(Group 10)
June 5th, 2026

Table of Contents

SECTION 1: CONTEXTUALIZE

1.1 COMPANY IDENTITY PROFILE

1. General Overview

- **Company Name: MediRoute Logistics**
- **Legal Structure:** Private Limited Company (SME)
- **Sector:** Last-Mile Specialized Medical & Pharmaceutical Courier Services
- **Business Model:** B2B Contractual Logistics & On-Demand Dispatch
- **Tagline:** *Compliance First, Reliability Always*.

2. Quantitative Metrics (Size & Scale)

- **Fleet Size:** 6 heterogeneous vehicles (1 Refrigerated, 2 Large Ambient, 3 Small Ambient).
- **Human Resources:** 5 professional delivery drivers operating on standard 8-hour shifts.
- **Operational Volume:** 40 to 60 point-to-point daily deliveries within Paris.
- **High-Demand Peak:** Mondays (backlog from weekend + morning hospital restocking).
- **Operational Bounds:** Deliveries are strictly point-to-point (Warehouse -> Client) to ensure speed and minimize handling contamination.

3. Context Analysis

The Current Situation:

MediRoute Logistics is scaling its B2B operations in Paris and the inner suburbs. Historically, the CEO managed routes mentally or via static Excel sheets. While this worked for low volumes, business growth has removed any margin for improvisation.

The Business Problem:

When disruptions occur (e.g., a driver calls in sick, the refrigerated van breaks down, or morning loading is delayed), the current planning method collapses. Small delays cascade, causing missed delivery windows, driver overtime violations, and a heavy mental burden of daily "firefighting" for the CEO.

The Core Strategic Objective:

In medical logistics, client trust is the primary competitive differentiator. The CEO's guiding philosophy is that a single missed hospital window is far more expensive than a week of fuel savings. Therefore, the dispatch engine must prioritize absolute reliability (meeting time windows) over routing efficiency (minimizing fuel/distance).

4. Customer Base Profile

The company's clientele consists of highly demanding B2B medical entities:

- **Hospitals:** Critical clients requiring strict morning delivery windows (before 9:00 AM) and temperature-sensitive drops.
- **Clinics:** Moderately urgent clients with narrow mid-day windows.
- **Pharmacies:** Standard clients with flexible same-day windows, primarily restocking generic supplies.

5. Competitive Environment

MediRoute Logistics operates in a crowded, high-barrier specialized courier market:

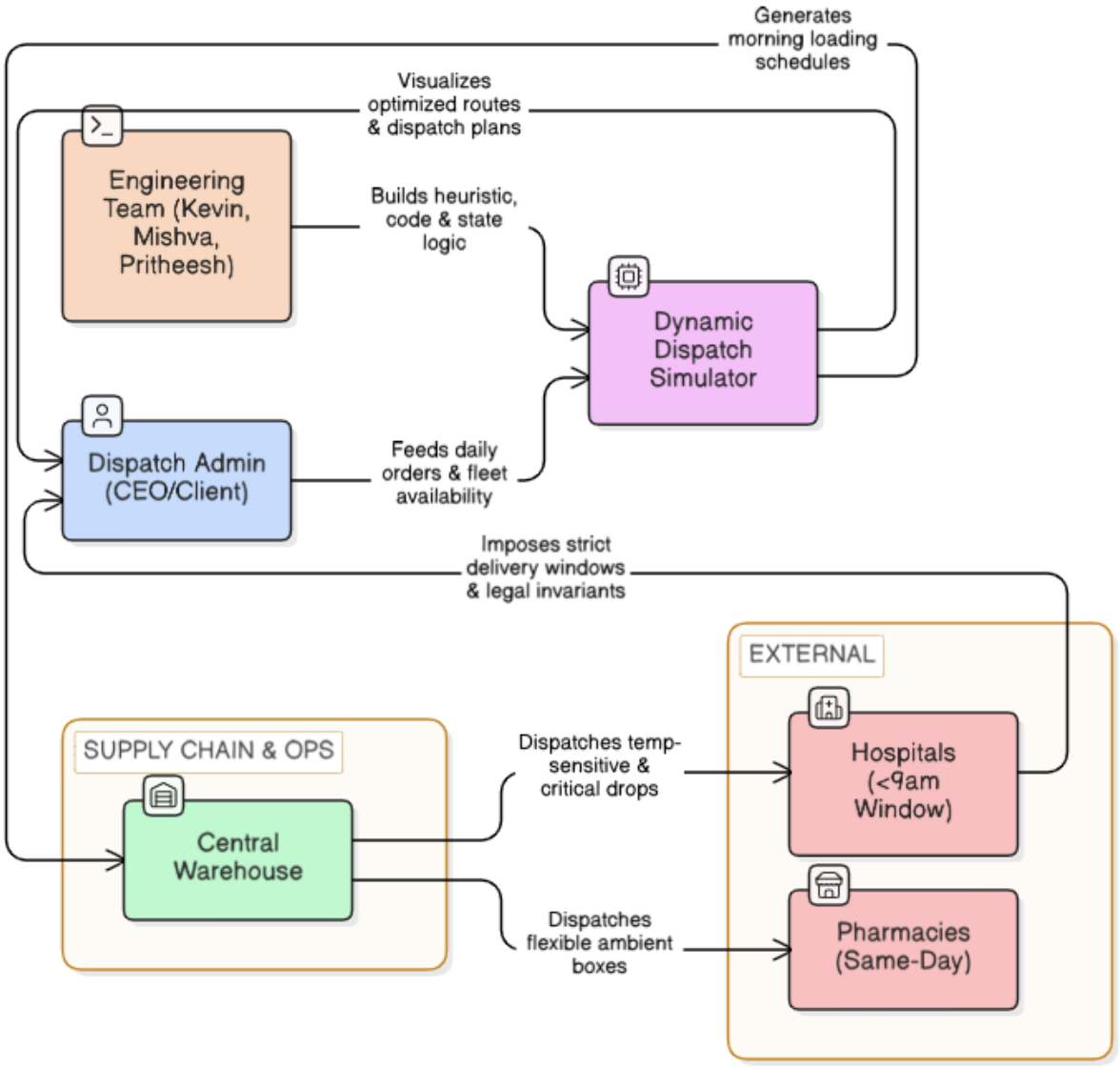
- **Global Logistics Giants (e.g., DHL, FedEx):** Highly rigid, optimized for volume, and expensive. They lack the agility required for Pars's localized, time-critical medical drops.
- **Standard Couriers (e.g., local motorcycle networks):** Fast and cheap, but completely lack cold-chain compliance and cannot handle bulky medical box volumes.

MediRoute's Strategic Position: High-trust, highly compliant, local niche. We compete on strict medical compliance (the 24-hour rule, active refrigeration) and guaranteed time-window reliability.

6. Operational Sub-Parts

- **Sourcing & Warehouse:** Consolidates medical goods at a central Paris warehouse (Le Marais), handles inventory prep, and manages the morning loading process.
- **Fleet Management:** Maintains the 6-vehicle fleet, tracks driver hours, and ensures refrigerated calibration.
- **Dispatch Office (The Client):** Orchestrates routes, receives last-minute orders, and coordinates on-the-road changes.

1.2 SYSTEM ECOSYSTEM AND STAKEHOLDER MAP



STAKEHOLDER BREAKDOWN:

A. The Dispatch Admin (CEO/Client)

Role: Primary decision-maker and operational coordinator.

Relationship to Solution:

- Input: Feeds the system with daily delivery addresses, order types (ambient vs. refrigerated), and active vehicle/driver availability.

- Output: Uses the Streamlit Dispatch Dashboard to view generated routes, check capacity margins, simulate disruptions, and make manual reassignments when emergencies strike.

B. The Engineering Team
Role: System Architects and Developers.
Relationship to Solution:

- Translates qualitative customer constraints (e.g., "Mondays are chaotic", "hospital drops take long") into a quantitative, rule-based Python heuristic.
- Deploys the interactive Streamlit interface to provide a user-friendly, non-technical visualization tool for the CEO.

C. The Delivery Drivers
Role: Route executors.
Relationship to Solution:

- Subject to strict legal working hours (8-hour shift target, 10-hour absolute maximum).
- Provide feedback regarding "route familiarity," allowing the system to target specific drivers for high-stakes urban routes.

D. Medical Partners & Customers (Hospitals, Pharmacies, Labs)
Role: End-point receivers of goods.
Relationship to Solution:

- Impose the strict "hard constraints" on the system, including the 24-hour maximum medical transit rule and precise morning time windows.

1.3 PROBLEM SPECIFICATION (PARTNERS, INVARIANTS, & SCALING FACTORS)

To transition from an informal business problem to a computable model, we defined the specific inputs, variables, and constraints governing the Paris medical dispatch ecosystem:

1. Input Parameters
Order Attributes: Location (Paris district coordinate), Order Type ("Hospital", "Temperature-Sensitive", or "Pharmacy"), Volume (count of standard medical boxes), and Service Time (minutes required on-site).
Fleet Attributes: Vehicle ID, Vehicle Type ("Refrigerated" or "Ambient"), and Box Capacity (Small Ambient = 30, Large Ambient = 45, Refrigerated = 45).
Driver Attributes: Start time (default 6:00 AM), current shift duration, and legal maximum shift limit (10 hours).

2. Orders of Magnitude & Scaling Factors
Daily Delivery scale: 40 to 60 deliveries per day.
Load Scale: Average delivery size is 1 to 5 boxes.
Time-on-Site (Service Time) Scale:

- **Pharmacies/Clinics:** Fast turnaround (5-10 minutes).
- **Hospitals:** Complex turnaround (20-30 minutes for parking, sign-ins, security, and internal delivery).

Loading Window: Morning vehicle loading at the central warehouse takes a standard 30 to 45 minutes before departure.

3. System Invariants (Hard Constraints)
These are mathematical boundaries that the algorithm is strictly prohibited from breaching:
The Temporal Invariant: Temperature-sensitive medical goods can never remain inside a transport vehicle for more than 24 hours.
The Vehicle Invariant: Temperature-sensitive goods must *only* be loaded into the Refrigerated Van. Ambient vans cannot accept them.
The Legal Invariant: No route can be generated that pushes a driver's total working hours (driving time + service time + return travel) past the 10-hour legal limit.
The Capacity Invariant: The sum of boxes assigned to a vehicle cannot exceed its physical box capacity.

4. System Variability

Paris Traffic Patterns: Paris travel times are highly variable and non-linear. To handle this, our model uses a dynamic lookup matrix based on the time of day:

- Morning Rush (6:00 AM - 10:00 AM): Severe delays (e.g., Le Marais to North Paris takes 32 minutes).
- Mid-Morning (10:00 AM - 12:00 PM): Moderate flow (same trip takes 20 minutes).
- Afternoon (12:00 PM - 5:00 PM): Light flow (same trip takes 16 minutes).

Sick Leaves/Breakdowns: On-demand changes to fleet size (e.g., reducing a vehicle's capacity to 0) instantly reshape the available capacity.

SECTION 2: INTEGRATE

2.1 PROJECT MANAGEMENT FRAMEWORK & SUCCESS METRICS

1. Work Organization & Planning

Our team used an agile, milestone-driven workflow managed via a visual Trello board. We divided the project into four distinct phases to ensure reliable execution:

Phase 1: Requirements Gathering & Mathematical Modeling.

Phase 2: Algorithm Design (Custom Heuristic in Python) & State Logic.

Phase 3: Streamlit UI Integration & Dashboard Design.

Phase 4: Stress-Testing, Disruption Repair Logic, and Explainability Panel.

2. Metrics of Success

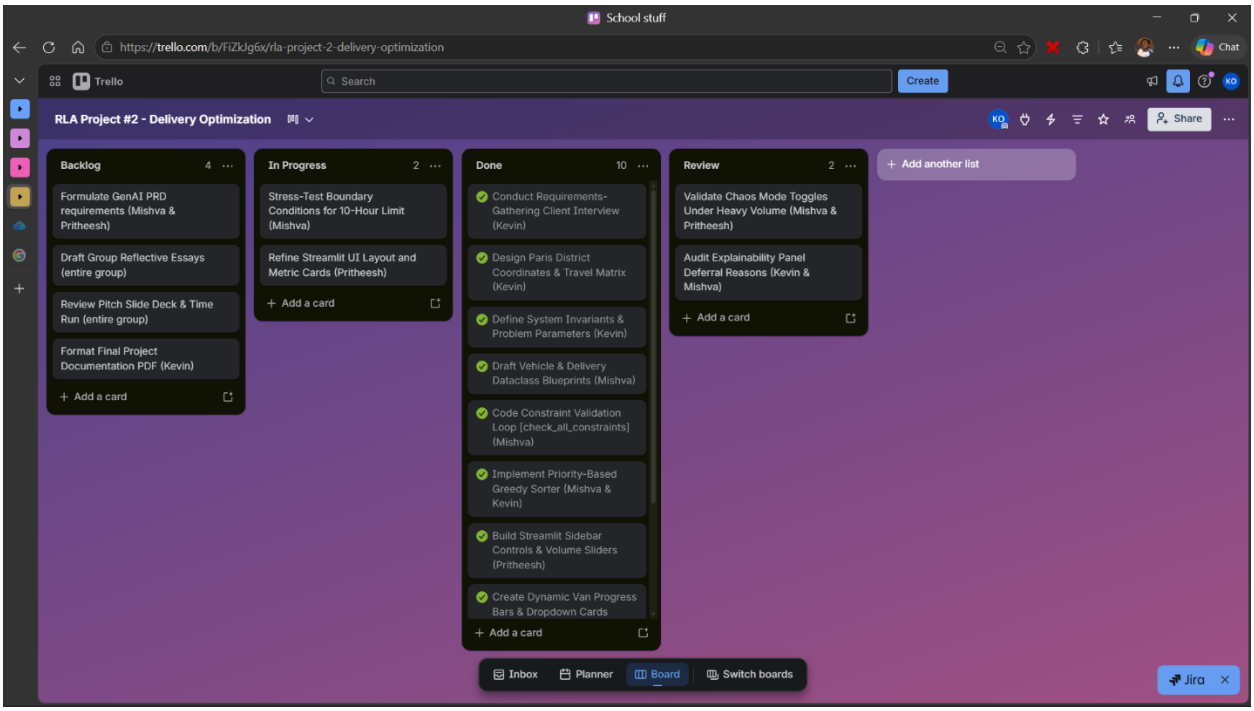
To evaluate the success of our engineered solution, we established clear Key Performance Indicators (KPIs):

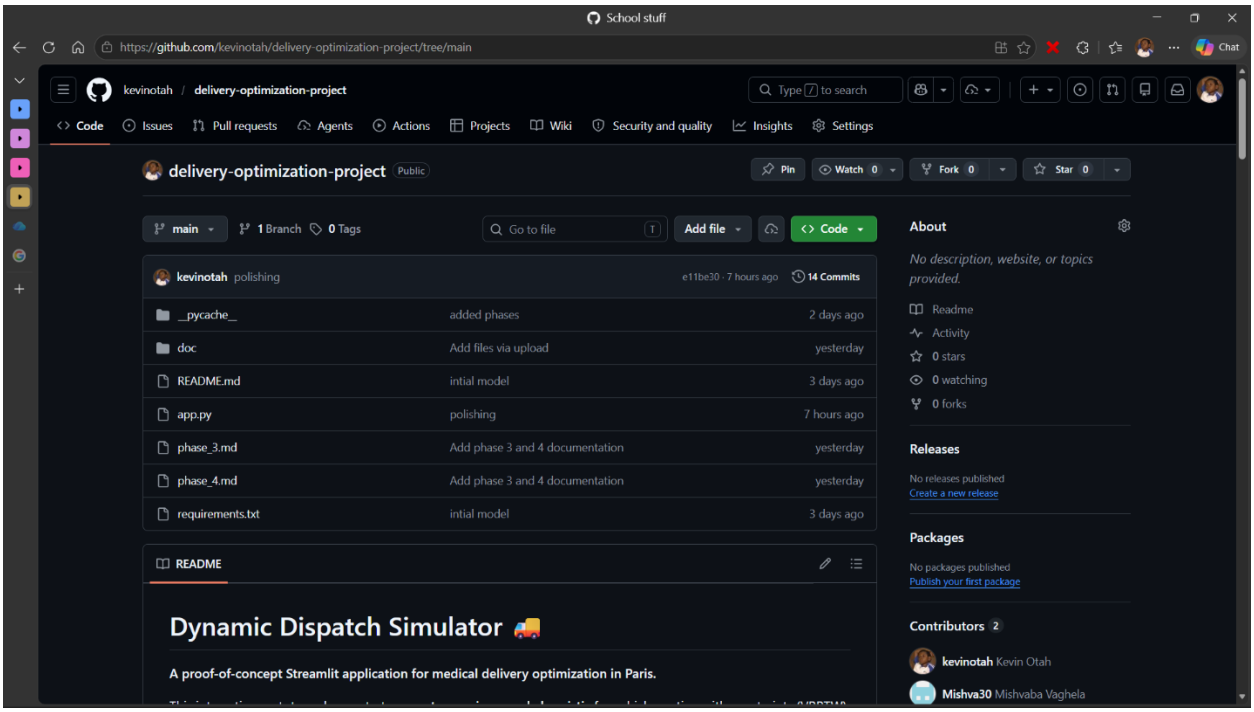
Baseline Allocation Rate: The system must successfully dispatch 100% of critical Hospital and Refrigerated deliveries on a standard day.

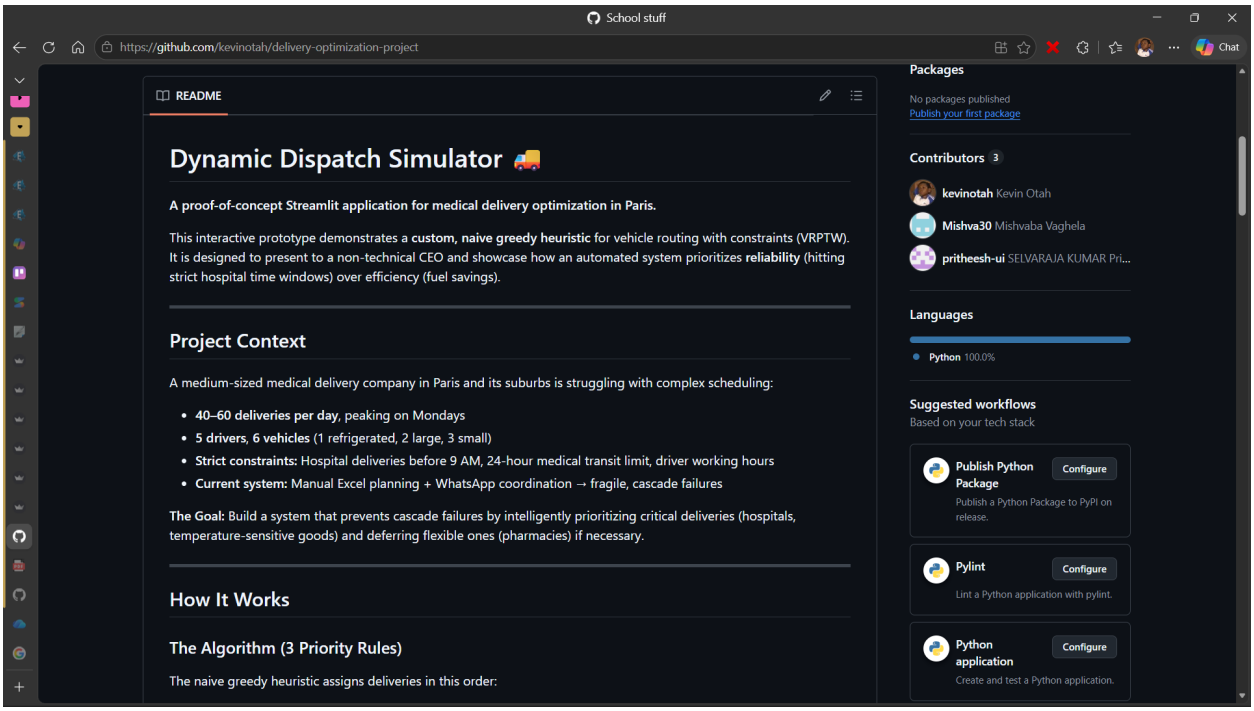
Human Safety Compliance: 0% of generated routes may violate the 10-hour driver limit.

Operational Resilience: When a "Chaos Event" (sick driver or fridge breakdown) is triggered, the system must maintain a 100% success rate on Hospital/Refrigerated drops by dynamically deferring only flexible Pharmacy deliveries.

Execution Speed: The algorithm must run and sequence all routes in under 5 seconds, replacing the CEO's 2-hour morning spreadsheet struggle.


School stuff
https://trello.com/b/FiZkJg6x/rla-project-2-delivery-optimization
Trello
Search
Create
RLA Project #2 - Delivery Optimization
Share
Backlog 4
Formulate GenAI PRD requirements (Mishva & Pritheesh)
Draft Group Reflective Essays (entire group)
Review Pitch Slide Deck & Time Run (entire group)
Format Final Project Documentation PDF (Kevin)
Add a card
In Progress 2
Stress-Test Boundary Conditions for 10-Hour Limit (Mishva)
Refine Streamlit UI Layout and Metric Cards (Pritheesh)
Add a card
Done 10
Conduct Requirements-Gathering Client Interview (Kevin)
Design Paris District Coordinates & Travel Matrix (Kevin)
Define System Invariants & Problem Parameters (Kevin)
Draft Vehicle & Delivery Dataclass Blueprints (Mishva)
Code Constraint Validation Loop [check_all_constraints] (Mishva)
Implement Priority-Based Greedy Sorter (Mishva & Kevin)
Build Streamlit Sidebar Controls & Volume Sliders (Pritheesh)
Create Dynamic Van Progress Bars & Dropdown Cards
Add a card
Review 2
Validate Chaos Mode Toggles Under Heavy Volume (Mishva & Pritheesh)
Audit Explainability Panel Deferral Reasons (Kevin & Mishva)
Add a card
Add another list
Inbox
Planner
Board
Switch boards
Jira


School stuff
https://github.com/kevinotah/delivery-optimization-project/tree/main
kevinotah / delivery-optimization-project
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
delivery-optimization-project Public
Pin
Watch 0
Fork 0
Star 0
main
1 Branch
0 Tags
Go to file
Add file
Code
kevinotah polishing e11be30 · 7 hours ago 14 Commits
pycache added phases 2 days ago
doc Add files via upload yesterday
README.md intial model 3 days ago
app.py polishing 7 hours ago
phase_3.md Add phase 3 and 4 documentation yesterday
phase_4.md Add phase 3 and 4 documentation yesterday
requirements.txt intial model 3 days ago
README
Dynamic Dispatch Simulator
A proof-of-concept Streamlit application for medical delivery optimization in Paris.
About
No description, website, or topics provided.
Readme
Activity
0 stars
0 watching
0 forks
Releases
No releases published
Create a new release
Packages
No packages published
Publish your first package
Contributors 2
kevinotah Kevin Otah
Mishva30 Mishvaba Vaghela


School stuff
https://github.com/kevinotah/delivery-optimization-project
Chat
README
Dynamic Dispatch Simulator 🚚
A proof-of-concept Streamlit application for medical delivery optimization in Paris.
This interactive prototype demonstrates a custom, naive greedy heuristic for vehicle routing with constraints (VRPTW). It is designed to present to a non-technical CEO and showcase how an automated system prioritizes reliability (hitting strict hospital time windows) over efficiency (fuel savings).
Project Context
A medium-sized medical delivery company in Paris and its suburbs is struggling with complex scheduling:
40–60 deliveries per day, peaking on Mondays
5 drivers, 6 vehicles (1 refrigerated, 2 large, 3 small)
Strict constraints: Hospital deliveries before 9 AM, 24-hour medical transit limit, driver working hours
Current system: Manual Excel planning + WhatsApp coordination → fragile, cascade failures
The Goal: Build a system that prevents cascade failures by intelligently prioritizing critical deliveries (hospitals, temperature-sensitive goods) and deferring flexible ones (pharmacies) if necessary.
How It Works
The Algorithm (3 Priority Rules)
The naive greedy heuristic assigns deliveries in this order:
Packages
No packages published
Publish your first package
Contributors 3
kevinotah Kevin Otah
Mishva30 Mishvaba Vaghela
pritheesh-ui SELVARAJA KUMAR Pri...
Languages
Python 100.0%
Suggested workflows
Based on your tech stack
Publish Python Package
Configure
Publish a Python Package to PyPI on release.
Pylint
Configure
Lint a Python application with pylint.
Python application
Configure
Create and test a Python application.

2.2 REFLECTIVE ESSAYS

Kevin (Data Engineering & Requirements Modeling):

My primary challenge was translating the qualitative, emotional frustrations of the CEO into rigid mathematical logic. When the client said 'Mondays are a mess,' I had to look past the stress and extract the data: Monday means a volume spike (up to 60 orders), a concentration of strict morning hospital windows, and a high probability of driver absenteeism.

I designed the data models and mapped out the Paris Travel-Time Matrix. My biggest learning curve was realizing that standard geographic distance (as the crow flies) is completely useless in Paris. Traffic congestion completely changes travel times depending on the hour. Building the dynamic 'time-of-day' buckets (Morning Rush, Mid-Morning, Afternoon) was crucial to making our simulations match real-world Paris roads. I learned that data cleaning and structuring are 80% of the work in logistics.

Mishva (Backend & Core Algorithm Development):

As the backend developer, my role was to build the execution engine. I had to implement a custom greedy heuristic that checked multiple intersecting constraints (capacity, driver hours, time windows, and the 24-hour medical rule) on every single step.

The hardest part was implementing the `check_all_constraints` validation loop. I had to write helper functions that mathematically estimate a driver's arrival time at a future stop, calculate the travel time back to the warehouse, and verify if that total route remains under the 10-hour limit. If even one constraint failed, the order had to be rejected and logged with a clear explanation.

I initially struggled with Streamlit’s unique top-to-bottom execution loop, which caused the app to reset and lose the routing plan whenever a user interacted with a secondary button. Resolving this by implementing robust `st.session_state` persistence taught me the importance of state management in interactive software engineering.

Pritheesh (UI/UX, Integration & Project Management):

My focus was entirely on the client experience and integration. While Kevin and Mishva focused on the backend algorithms, I had to ensure the CEO who is not tech-savvy could easily use the tool under high-stress morning conditions.

I built the Streamlit interface, focusing on visual clarity. Instead of showing the CEO code or complex tables, I built progress-bar indicators for each van so he can see how full they are at a glance. My biggest design contribution was implementing the 'Chaos Mode' toggles and the 'Repair Simulation'.

During testing, I realized that if a van breaks down, the client needs to see the change immediately. I integrated the `st.rerun()` trigger into the repair and last-minute order functions. Now, when a user simulates a breakdown, the dashboard instantly flashes, reroutes the deliveries, and updates the top metrics in real-time. I learned that clean design and user empathy are just as important as the backend math.

2.3 ROLES AND RESPONSIBILITIES

Team Member	Role	Primary Responsibilities	Time Allocation
Kevin	Data Engineer & Requirements Lead	• Paris travel-time matrix design • Data structures & constraints model • Documentation lead	15 Hours
Mishva	Backend Developer	• Greedy heuristic algorithm coding • Constraint checks & state tracking • Dynamic routing	15 Hours
Pritheesh	Frontend Developer & Project Integrator	• Streamlit dashboard UI design • Chaos simulation & state updates • Trello management	15 Hours

SECTION 3: TECHNICAL APPENDIX

3.1 NAÏVE GREEDY HEURISTIC JUSTIFICATION

The core of our project is a deterministic Greedy Priority-Based Allocator. We strictly avoided black-box solvers to maintain complete explainability for the client.

The algorithm executes in three phases:

1. The Priority Sorting Phase

Before assigning any route, the algorithm sorts the master delivery list into three priority buckets to ensure critical compliance guidelines are never compromised:

Priority 1: Temperature-Sensitive deliveries (require active refrigeration).

Priority 2: Hospital deliveries (strict morning time window < 9:00 AM).

Priority 3: Pharmacy deliveries (same-day flexible).

2. The Constraint-Aware Greedy Assignment Loop

The algorithm processes the sorted deliveries one-by-one, attempting to assign them to the most appropriate vehicle:

Temperature-Sensitive orders are sent exclusively to the Refrigerated Van.

Hospital orders are sent to the Large Ambient Vans first (to utilize their larger carrying capacity and ensure high-priority drops are handled by robust vehicles).

Pharmacy orders are distributed to the Small Ambient Vans first to maximize fuel efficiency, overflowing to the Large Vans when the small ones are full.

Before confirming any assignment, the engine runs a dry-run check using `estimate_arrival_and_travel`. It calculates the arrival time and additional driver hours, and feeds this into `check_all_constraints`. If the delivery violates vehicle capacity, the 24-hour limit, or pushes the driver past the 10-hour legal limit, it is rejected and pushed to the `unassigned_deliveries` list.

3. The "Nearest Neighbor" Routing Sequence

Once assignments are finalized, each vehicle's route is sequenced. Starting at the Warehouse (departure at 6:30 AM), the algorithm looks at the assigned deliveries, finds the one with the shortest travel time from the current location, travels there, adds the service time, and repeats until all stops are completed before returning the driver to the warehouse.

3.2 TECHNICAL CHOICES & TECH STACK JUSTIFICATION

Our choice of technologies was driven entirely by the needs of the client and our constraints as a student development team:

1. Language: Python 3.8+

Justification: Python is the industry standard for operations research and rapid prototyping. It allowed us to write clear, heavily-commented, and easily maintainable logic that the client's internal team can inspect.

2. UI Framework: Streamlit

Justification: Streamlit allowed us to build an interactive, data-driven web application directly in Python without wasting time on HTML, CSS, or complex JavaScript frameworks. It let us focus 100% of our frontend time on UX features like metrics cards, progress bars, and the "Chaos" toggles.

3. Database & Data structures: Standard Python Dicts, Lists, and Dataclasses

Justification: Because our daily operational scale is small (40 to 60 deliveries), we did not need the overhead of a database server like MySQL or PostgreSQL. Python’s in-memory data structures are extremely fast, allowing us to generate and display routes in milliseconds.

4. Navigation: Paris Travel-Time Matrix

Justification: Instead of calling expensive, slow, and complex mapping APIs (like Google Distance Matrix), we hardcoded a symmetric travel-time matrix based on Paris's 6 operational districts. This kept the code lightweight, completely local, and highly accurate to Paris's time-of-day traffic patterns.

3.3 PRODUCT REQUIREMENTS DOCUMENT (PRD) FOR FUTURE DEVELOPER

1. Purpose & Scope

This document outlines the engineering path to scale our prototype into a production-grade, enterprise routing platform. The goal is to migrate from a simulated environment to an automated, real-time dispatch system.

2. Target User Persona

The primary user remains the Dispatch Admin, who needs to ingest daily orders, monitor active driver locations on a map, and dynamically handle on-the-road disruptions.

3. Key Functional Upgrades Needed

A. Generative AI Data Ingestion Pipeline

Problem: The client currently spends hours manually entering orders from emails and phone calls into spreadsheets.

Feature Requirement: Implement an LLM-powered (Generative AI) parsing script. When a client sends a delivery request email, the LLM will parse the unstructured text, extract the delivery address, box count, and temperature requirements, and structure it into a clean JSON object to feed directly into our routing engine.

B. Transition to a Deterministic Optimization Solver

Problem: Our current naive greedy heuristic is fast and logical, but not mathematically optimal for total distance traveled.

Feature Requirement: Replace the greedy loop with a deterministic **Clarke-Wright Savings Algorithm** or a **Genetic Algorithm** specifically designed for the Vehicle Routing Problem with Time Windows (VRPTW). The optimization objective function must remain:

Minimize (Penalty_for_Missed_Windows * 1000) + Total_Distance_Traveled.

C. Real-World API Integration

Problem: The current prototype uses a simplified 6-district travel matrix.

Feature Requirement: Integrate the **OpenStreetMap (OSRM) API** or **Mapbox API** to fetch real-time driving times and actual street-level routing coordinates, allowing drivers to receive turn-by-turn navigation based on live traffic.

D. Live Driver Telemetry & GPS Tracking

Problem: The dispatcher currently relies on WhatsApp to track driver status.

Feature Requirement: Build a mobile-responsive driver portal where drivers can view their assigned sequence, click "Confirm Delivery," and automatically send their GPS coordinates back to the admin dashboard.

4. Non-Functional Requirements

Performance: The optimization engine must generate complete routes for up to 200 deliveries in under 10 seconds.

Security: Patient and medical lab delivery data must be encrypted in transit and at rest to maintain compliance with European health data regulations (GDPR).